

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Kinds in Haskell and Their Analogues in F# and JavaScript

Relevant prompts

Course:

Advanced Programming

Student:

Claudio Bernardoni

617214

ACADEMIC YEAR 2025/2026

Contents

1 Adopted approach **2**

2 Relevant AI Prompts **2**
Prompt 1 — First draft for Section 1 2
Prompt 2 — Refinement of the initial draft for Section 1 3
Prompt 3 — Review phase of my Section 1 corrections 5
Prompt 4 — First draft for Section 2 5
Prompt 5 — Review phase of my Section 2 corrections 8
Prompt 6 — First draft for Section 3 9
Prompt 7 — Review phase of my Section 3 corrections 11
Prompt 8 — Conclusion 13
Prompt 9 — Final review 14

3 Claude context **17**
3.1 C10 - Kinds in Haskell and Their Analogues in F# and JavaScript 17
3.1.1 Goal 17
3.1.2 Tasks 17
3.2 Previous knowledge 17
3.3 Project structure 17
3.3.1 AI output 17

1 Adopted approach

The development of this project involved the use of multiple generative AI models, each assigned a specific role within the workflow. Rather than relying on a single model to produce the entire manuscript, the process was designed to take advantage of the different capabilities of each tool. Particular attention was given to the quality of the prompts and to the continuous evaluation of the generated material. No output was accepted without review, and each stage of the process involved direct supervision, critical assessment, and, when necessary, further refinement.

Before beginning the generation of the manuscript, some preliminary research into *prompt engineering* was conducted to better understand how to formulate effective instructions for large language models. This included studying how to provide sufficient context, define clear objectives, specify constraints, and structure requests to obtain more accurate and consistent results.

Three AI models were used throughout the project, each serving a distinct purpose:

- **ChatGPT 5.6 Sol**: Used primarily to develop, structure, and refine the prompts subsequently submitted to Claude. It was also used for assistance with LaTeX formatting, document organization, and pagination.
- **Claude Opus 5.5**: Used as the primary content-generation model, responsible for producing the initial drafts of the manuscript and subsequently revising them according to the feedback and corrections provided. It was also provided with a persistent project context specifying the objectives, constraints, and expected structure of the report, which can be found in Section 3
- **Gemini 3.6 Reasoning**: Used to support independent research, particularly in identifying relevant documentation, technical references, and other sources against which the generated explanations could be checked. The sources identified through Gemini were then consulted directly rather than accepting its summaries as sufficient evidence.

The manuscript was developed through an iterative workflow applied independently to each of its four sections. For each section, the process began with the preparation of a detailed prompt using ChatGPT. Once finalized, it was submitted to Claude, which generated an initial draft of the corresponding section.

The generated material was then carefully read and critically evaluated. During this review, potentially inaccurate statements, unclear explanations, questionable examples, and opportunities for improvement were identified. When additional information was needed, Gemini was used to locate relevant sources, which were subsequently examined to verify the correctness of the material. The draft was annotated with observations, questions, and proposed corrections, and the annotated version was then submitted back to Claude for revision.

This cycle of *generation, review, research, annotation, and revision* was repeated until the section reached a satisfactory level of correctness, clarity, and completeness. The same procedure was followed for all five sections of the manuscript.

2 Relevant AI Prompts

The following sections document the most relevant prompts used during this process.

> **Prompt 1 — First draft for Section 1**
>
> You are my personal assistant for an Advanced Programming university project about kinds in Haskell and their analogues in F# and JavaScript/TypeScript.
> **Context:** This is the introductory section of the project and should focus exclusively on Haskell. You may assume that the reader has good basic knowledge of functional programming and Haskell, including types, polymorphism, algebraic data types, and type constructors, but has no prior knowledge of

kinds.
Task: Write an introductory but technically rigorous section explaining Haskell's kind system and why kinds are useful. The section should:

1. Introduce the distinction between values, types, type constructors, and kinds, motivating why a level above types is necessary.
2. Define kinds and explain the traditional * notation as well as the modern GHC Type notation.
3. Explain progressively the kinds of ordinary types and type constructors, including examples such as `Int`, `Bool`, `Maybe`, `Either`, and `[]`.
4. Explain kinds such as `Type`, `Type -> Type`, and `Type -> Type -> Type`, including how partial application of type constructors changes the resulting kind.
5. Explain what is meant by higher-kinded types/type constructors in Haskell and why the concept is useful.
6. Explain how the Haskell compiler uses kinds to check whether types and type constructors are combined correctly. Include at least one example of a valid type-level application and one example that would produce a kind mismatch.
7. Introduce what "type-level code" means in this context and explain the role of kind checking at the type level. This discussion should be substantial enough for the reader to understand what the compiler is checking, but it should remain part of the broader introduction rather than becoming the sole focus of the section.
8. Explain the practical usefulness of kinds, rather than presenting them only as a formal classification system.

Advanced features such as DataKinds, PolyKinds, kind inference, or other GHC extensions may be mentioned briefly where they clarify the discussion, but they should not dominate this introductory section. Organize the explanation so that each concept naturally motivates the next one. Prefer concrete Haskell examples before introducing more abstract terminology.
Format: Return a self-contained section of a technical report written in LaTeX. Use appropriate sections/subsections, prose, mathematical notation where useful, and properly formatted Haskell code examples.
Rules:

- Cite all substantive technical claims.
- Prefer authoritative primary sources, especially the GHC User's Guide, the Haskell Report where applicable, and reputable academic literature.
- Do not invent or guess references.
- Make clear which source supports each important technical point.
- Provide complete bibliographic information or links sufficient for every cited source to be independently verified.
- Do not assume that the reader already knows what kinds are.
- Keep the treatment rigorous but introductory: explain the reasoning behind kinds without making the section unnecessarily formal or pedantic.

Prompt 2 — Refinement of the initial draft for Section 1

Role: You are editing an existing section of my Advanced Programming university project.
Context: I have attached the current version of Section 1, "Kinds in Haskell". Its technical quality and general explanatory progression are good, but it is much too long and its treatment of sources is inappropriate for the final report.
The complete project will be approximately 30 pages, so Section 1 should occupy only around 4–5 pages.
Task: Reformulate the attached Section 1 rather than writing an unrelated replacement.

Preserve the strongest parts of the existing explanation and its overall pedagogical progression, but substantially compress it.
The new version should retain:

- the motivation for introducing kinds;
- the distinction between values, types, type constructors, and kinds;
- `Type`, `Type -> Type`, and `Type -> Type -> Type`;
- examples involving `Maybe`, `Either`, and `[]`;
- partial application of type constructors;
- higher-kinded types, using `Functor` as the principal example;
- how kind checking works;
- one or two representative kind errors;
- a concise explanation of type-level code;
- the practical usefulness of kinds.

Compress or remove:

- repeated explanations of the same concept;
- long historical or notation discussions;
- detailed treatment of GHC-version-specific behaviour;
- long discussions of `DataKinds`, `PolyKinds`, type families, levity polymorphism, or similar advanced topics;
- excessive examples where one representative example would suffice;
- material whose main purpose is to document the source rather than explain the concept;
- the separate summary subsection if the preceding exposition already provides a natural conclusion.

Do not merely delete paragraphs mechanically. Rewrite transitions and reorganize material where needed so that the shorter version remains coherent.
Source handling: The final report must contain NO explicit discussion of its sources.
Remove all formulations such as:

- "The Report states..."
- "The Report defines..."
- "The Report itself uses..."
- "According to the Haskell Report..."
- "The GHC User's Guide says..."
- "the documentation gives..."
- or any equivalent source-oriented wording.

Technical claims supported by those sources may remain, but they must be stated directly as facts.
Produce THREE files:

1. 'section1.tex'
   - the rewritten 4–5 page report section;
   - no citations or bibliography;
   - no references to external documents or documentation in the prose.
2. 'section1-sources.tex'
   - a separate verification document;
   - list the sources used for the rewritten section;
   - group sources by the claims/concepts they support;
   - provide section numbers, URLs, DOIs, or other precise locations where possible;
   - this file will not be included in the final report.
3. 'main.tex'
   - a complete compilable LaTeX entry point for 'section1.tex';
   - include all required packages;
   - define every macro, environment, color, listing style, and custom command used by 'section1.tex';
   - use '\inputsection1';

- do not leave any dependency implicit.

Important: Do not introduce new advanced material merely because space becomes available after shortening the report. The goal is to reduce the section from approximately 12 pages to approximately 4–5 pages while retaining its conceptual quality.

Prompt 3 — Review phase of my Section 1 corrections

I have annotated the attached report with review comments. Do not rewrite the section from scratch. First, inspect every annotation individually. For each annotation:

1. briefly explain whether you agree, partially agree, or disagree;
2. explain the technical reason;
3. state exactly what change, if any, you would make.

Do not automatically accept my comments: if one of my proposed corrections is technically wrong or would make the report worse, explain why and preserve the correct version. Then produce a revised section1.tex incorporating the accepted changes while preserving the existing structure, style, length, and material wherever possible. In particular, prefer local corrections and rewrites rather than regenerating the section. Keep the result at approximately its current length of five / six pages.

Prompt 4 — First draft for Section 2

Role:
You are my personal assistant for an Advanced Programming university project about kinds in Haskell and their analogues in F# and JavaScript/TypeScript. You are an expert in type systems and functional programming.
You are writing for an advanced computer science student who already understands all the concepts introduced in Section 1 of the project, including Haskell kinds, type constructors, partial application, higher-kinded polymorphism, Functor, kind checking, and type-level code.
Context:
This is Section 2 of the project. It is a deep dive into the F# type system, with the specific goal of understanding to what extent Haskell's notion of kinds has an analogue in F#, what F# supports instead, what is missing, and how programmers work around those limitations.
The attached Section 1 should be used as the conceptual reference point for the comparison. Do not re-explain Haskell kinds from scratch. Whenever useful, reuse the same conceptual progression and comparable examples so that differences between the two languages are easy to see.
Task:
Write a technically rigorous section explaining how the F# type system relates to Haskell's kind system.
The main question of the section should be:
"To what extent can F# express and reason about the same abstractions that Haskell expresses through kinds and higher-kinded types?"
Structure the discussion around concrete comparisons rather than around a generic survey of the F# type system.
The section should cover the following points:

1. Briefly introduce only the aspects of the F# type system that are necessary for the comparison:
   - generic types and generic type parameters;
   - generic type definitions such as `option<'T>`, `list<'T>`, and `Result<'T,'E>`;
   - type inference and relevant generic constraints. Do not spend substantial space
   - explaining basic concepts already familiar to an advanced programming student.

2. Compare ordinary F# generic type constructors with the Haskell examples introduced in Section 1:
   - `option<'T>` with `Maybe`;
   - `list<'T>` with `[]`;
   - `Result<'T,'E>` with `Either`. Explain what sense, if any, it makes to think of these F# generic type definitions as corresponding to Haskell type constructors of kinds such as `Type -> Type` or `Type -> Type -> Type`.
3. Explain whether F# has an explicit kind system comparable to Haskell's. In particular, explain what the F# compiler checks when generic types are applied:
   - generic arity;
   - valid type application;
   - constraints on type parameters;
   - other compile-time restrictions that play a role analogous to part of Haskell's kind checking. Clearly distinguish between a genuine kind system and mechanisms that merely have a similar practical effect.
4. Investigate partial application of type constructors. Use Haskell's `Either String :: Type -> Type` as the reference example and compare it with F#'s `Result<'T,'E>`. Explain:
   - whether a generic F# type can be partially applied and treated as a new first-class type constructor;
   - what can instead be achieved with a type abbreviation such as a one-parameter alias of `Result`;
   - why this is or is not equivalent to Haskell's partial application.
5. Make higher-kinded polymorphism the central comparison. Explain whether F# can declare a generic parameter corresponding to Haskell's:
   `f :: Type -> Type`
   and use that parameter abstractly as in:
   `fmap :: (a -> b) -> f a -> f b`
   Explain precisely what F# can and cannot express here.
6. Use Functor/map as the main practical example. Compare Haskell's single Functor abstraction with F# functions such as:
   - List.map;
   - Option.map;
   - Result.map.

   Explain why these operations have the same structural pattern, but why expressing one ordinary generic map abstraction over all such type constructors is different in F#.
7. Explain the principal workarounds used in F# when higher-kinded polymorphism would otherwise be useful. Investigate techniques such as:
   - statically resolved type parameters and inline functions;
   - member constraints / trait-style constraints;
   - explicit witness or encoding techniques;
   - relevant library abstractions, if useful.

   For each workaround, explain:
   - what Haskell capability it is approximating;
   - how it works at a high level;
   - what limitations or trade-offs remain.

   Do not let these techniques dominate the section: they should follow from the previously established limitation rather than becoming an independent survey of advanced F# features.
8. Briefly discuss how these differences affect library design in F#. In particular, explain why F# often exposes operations separately for concrete containers (`List.map`, `Option.map`, etc.) and how this relates to the absence or limited expressibility of higher-kinded abstractions.
9. End with a concise comparison that makes clear:

- what F# has that is directly analogous to Haskell type constructors;
- what is only superficially similar to Haskell kinds;
- what F# cannot express directly;
- which mechanisms are commonly used to recover some of that expressive power.

Throughout the section, prefer side-by-side Haskell/F# examples when they clarify an important difference. Do not repeat the full explanation of Haskell concepts already provided in Section 1.
Advanced F# features may be discussed when they directly help answer the comparison above, but they must not become the main subject of the section.
Organize the explanation so that each limitation naturally motivates the workaround discussed next.
Format:
You must return three files:

1. section2.tex
   - Contains Section 2 of the report.
   - Use the macros and environments already defined in the attached main.tex where appropriate.
   - Do not reproduce definitions of macros that already exist.
   - The section should integrate naturally after Section 1.
2. sources2.tex
   - This file is only for verification and will not be part of the report.
   - For every substantive technical claim in section2.tex, provide an authoritative source that I can use to verify it.
   - Group the sources by topic or claim rather than merely providing a bibliography.
   - Give precise section names, URLs, documentation pages, papers, or other locations sufficient for independent verification.
   - Prefer official Microsoft/F# documentation, the F# language specification, relevant .NET documentation, academic literature, and authoritative library documentation.
3. macros2.tex
   - Contain only new macros, environments, packages, or formatting definitions that
   - Section 2 requires and that are not already available in the attached main.tex.
   - Do not duplicate existing definitions.

Rules:

- Focus exclusively on F# and its comparison with the Haskell concepts established in Section 1.
- Do not turn the section into a general introduction to F#.
- Do not claim that F# "has kinds" unless this is technically justified; distinguish carefully between actual kinds and analogous compiler mechanisms.
- Make higher-kinded polymorphism and the inability or ability to abstract over generic type constructors a central part of the comparison.
- Use concrete examples before introducing advanced workaround techniques.
- Find an authoritative reference for every substantive technical claim.
- References and source-oriented discussion must appear only in sources2.tex, never in the prose of section2.tex.
- Do not write phrases such as "the F# specification states...", "Microsoft documentation says...", or similar source-oriented wording in the report.
- Aim for approximately 7–9 compiled pages.
- Prefer simple LaTeX over unnecessary custom macros.
- Avoid repetition and excessive subdivision.
- Do not add advanced topics merely to increase completeness or length.

Prompt 5 — Review phase of my Section 2 corrections

Role:
You are my technical reviewer for an Advanced Programming university project comparing Haskell kinds with F# and JavaScript/TypeScript.
You have expertise in functional programming, type systems, Haskell, F#, and the .NET type system. Your task is to evaluate my annotations impartially, not to defend the existing report or automatically agree with my proposed corrections.
Context:
I have attached the current version of Section 2, "Kinds and the F# Type System", with my annotations embedded in the PDF.
The section is generally good, and I do not want it recreated from scratch. I want to identify genuine technical errors, misleading explanations, unclear passages, and unnecessary material while preserving the existing structure and conceptual progression wherever possible.
Section 1 has already introduced Haskell kinds, type constructors, partial application, higher-kinded polymorphism, Functor, and kind checking. Section 2 should build on those concepts rather than explain them from scratch.
Task:
Review every annotation in the attached PDF individually. For each annotation, identify the associated passage and evaluate my comment impartially.
Classify each annotation using exactly one of these three outcomes:

- **Agree:** My annotation identifies a genuine problem, and the proposed direction is appropriate.
- **Partly agree:** My concern is justified, but my explanation or proposed correction needs qualification or modification.
- **Disagree:** The passage is technically correct or my proposed correction would introduce a problem. Explain why.

For every annotation, provide:

1. **The passage concerned:** Quote or identify the relevant sentence, code example, table, or paragraph.
2. **Your verdict:** Agree, partly agree, or disagree.
3. **Reasoning:** Explain the technical and/or pedagogical reason for your verdict. Do not give a superficial answer.
4. **Recommended correction:** State exactly what you would change, including replacement wording or corrected code where useful. If no change is needed, say so explicitly.
5. **Verification:** For substantive technical disputes, consult authoritative sources and provide the exact documentation section, URL, or other location that supports your conclusion. Distinguish between what a language specification guarantees, what a particular compiler version accepts, and what is merely conventional practice.

Do not automatically accept my comments. In particular, investigate rather than assume the answers to questions involving Haskell type synonyms and partial application, F# generic constraints, FSharpPlus, statically resolved type parameters, and the distinction between kinds and other compile-time restrictions.
Additional review:
After assessing all my annotations, inspect the rest of the section for important problems that I did not mark.
Look especially for:

- technically incorrect or overgeneralized claims;
- code examples that do not compile or that are presented as executable despite being schematic;
- misleading analogies between Haskell and F#;
- statements that confuse kind errors with generic arity, type constraints, or other compiler restrictions;
- incorrect or oversimplified explanations of F# workarounds for higher-kinded polymorphism;

- examples that obscure rather than demonstrate the intended comparison;
- unnecessary historical or advanced material;
- unsupported claims that should be verified or qualified.

Do not manufacture objections to minor stylistic choices. Prioritize issues that affect correctness, clarity, or the central argument of the section.

Output format:

Organize your response into three parts.

Part I — Assessment of my annotations

Provide a table with one row per annotation, in PDF order. Include the annotation's subject, verdict, main reason, and recommended action. Follow the table with detailed explanations wherever the technical issue warrants more discussion.

Part II — Additional issues

List the important issues you identified that are not covered by my annotations. For each, explain why it matters and propose a concrete correction.

Part III — Final output

Produce a new report that corrects the wrong parts.

Rules:

- Be impartial and technically rigorous. My annotations are hypotheses to evaluate, not instructions that must be followed.
- Do not rewrite the entire section.
- Preserve technically correct material even when I have questioned it.
- When a correction is warranted, provide concrete replacement wording or code rather than merely saying that something should be improved.
- Use authoritative sources wherever possible, especially the F# language specification, official Microsoft documentation, the GHC User's Guide, and relevant library documentation.
- Never invent a source, a compiler error, or a language rule.
- Keep the focus on the comparison between F# and Haskell kinds, particularly higher-kinded polymorphism and the workarounds used in F#.

Prompt 6 — First draft for Section 3

Role

You are an expert in programming-language type systems, Haskell, F#, JavaScript and TypeScript. You are writing Section 3 of my Advanced Programming university report about Haskell kinds and their analogues in F# and JavaScript/TypeScript.

The reader is an advanced computer science student who already understands the material presented in Sections 1 and 2 of the report.

Context

I have provided the project assignment and the existing report, which contains Sections 1 and 2.

Section 1 introduces Haskell kinds, type constructors, partial application, higher-kinded polymorphism, `Functor`, kind checking and type-level code.

Section 2 examines F# generics, the absence of native higher-kinded polymorphism, and workarounds including inline functions, member constraints and explicit encodings using `App<'F,'A>` and witnesses.

Treat these sections as established context. Refer back to their concepts where useful, but do not repeat their explanations unnecessarily. Maintain consistent terminology, notation, LaTeX style and level of technical detail.

Objective

Write a technically rigorous and self-contained Section 3 explaining how TypeScript can approximate some of the abstractions and reasoning enabled by Haskell's kind system, what TypeScript already provides, and what remains impossible or requires workarounds.

The central question should be:
How far can TypeScript approximate the abstractions enabled by Haskell's kinds, and what guarantees are lost when those abstractions are encoded rather than supported natively?
The emphasis must be on understanding the design and its limitations, not on building a large implementation.

Scope and structure

Organize the section so that each part naturally motivates the next. Use a small number of meaningful subsections rather than a long sequence of short ones.
Cover the following topics.

1. **TypeScript's existing type system.** Briefly explain generics using a relevant example. Distinguish TypeScript's static type system from JavaScript's runtime semantics, including the fact that TypeScript type annotations and type-level constructs do not generally survive compilation as runtime type information. Explain why ordinary generics do not automatically provide Haskell-style higher-kinded polymorphism.
2. **The connection with Haskell kinds.** Relate TypeScript's capabilities and limitations to the concepts introduced in Section 1: type constructors, partial application, higher-kinded variables, `Functor`, and kind checking. Distinguish features that TypeScript genuinely supports from patterns that can only be encoded indirectly. Avoid repeating the full explanation of Haskell's kind system.
3. **One small, illustrative encoding.** Present a minimal TypeScript encoding that approximates higher-kinded abstraction. Prefer a lookup-table or URI-to-type encoding, where a symbolic identifier represents a supported type constructor and a type-level mapping represents its application to an element type. Use a simple abstraction such as `Functor` and `map`, demonstrated with at most two containers, for example arrays and a minimal `Option` type. Include enough code to make the idea understandable and show how the abstraction can be used generically. Explain the role of every important type-level component. Identify exactly what the TypeScript compiler checks, what the encoding guarantees, and what remains a convention or requires manual registration. Do not implement a complete kind system, extend the TypeScript compiler, or build a general-purpose functional programming library. The example is intended to demonstrate one useful pattern, not to reproduce all of Haskell's type-level language.
4. **What the encoding cannot express.** Compare the encoding with native Haskell kinds, focusing on higher-kinded polymorphism, type-constructor application, partial application and the compiler's ability to reject ill-kinded expressions. Explain the practical limitations and costs of the encoding, such as boilerplate, the need to register supported constructors, and the lack of general native kind checking.
5. **Practical value and comparison with F#.** Use one or two short examples to explain when this abstraction is useful, such as writing a mapping operation once for multiple containers. Compare the TypeScript approach briefly with the relevant F# limitations and workarounds from Section 2. Explain what each approach recovers and what it sacrifices. Do not repeat the F# section.
6. **Compilation and runtime implications.** Clarify which parts of the example are checked statically, which parts remain ordinary JavaScript at runtime, and whether the encoding introduces any runtime machinery. Do not imply that TypeScript has acquired native kinds or that JavaScript performs kind checking.

Advanced TypeScript features may be discussed briefly when they clarify the comparison, but they must not become a separate survey or displace the main explanation.

Technical and verification requirements

- Keep the distinction between native language features, library-level encodings and runtime conventions explicit throughout.
- Do not describe a workaround as equivalent to Haskell's kind system unless the equivalence is carefully qualified.
- Explain both the benefits and the limitations of the proposed encoding.

- Ensure that every code example is internally consistent and valid TypeScript. Prefer examples that can be checked with a modern TypeScript compiler in strict mode.
- Do not use pseudocode without labelling it as such.
- Base descriptions of TypeScript, JavaScript, Haskell and F# behavior on authoritative documentation or other reliable technical sources.
- For every substantive, externally verifiable technical claim, identify a source that supports it. Distinguish documented language behavior from your own analysis of the proposed encoding.
- If a claim cannot be verified, qualify it or omit it rather than inventing a reference.
- Preserve the terminology and notation established in Sections 1 and 2.
- Do not introduce unrelated programming languages or extended discussions of advanced TypeScript features.

Length and style

The complete Section 3 must occupy **7–11 pages when incorporated into the report**, using the provided LaTeX setup. Do not aim to fill the maximum available space if not necessary.

Do not pad the section to meet the page requirement. If the draft is too long, prioritize the central explanation, the illustrative encoding, and the comparison of guarantees and limitations.

Prefer a small number of substantial subsections, concise explanations and short code examples. Avoid excessive subdivision, repetition, long digressions and unnecessarily elaborate implementations.

Deliverables

Return these three files:

`section3.tex` — the complete LaTeX source for Section 3. `sources3.tex` — a record of the sources used to verify the section's technical claims. `macros3.tex` — only the new LaTeX macros or environments genuinely required by Section 3.

Use the macros and environments already defined in the supplied `main.tex` and established in the existing report. Do not redefine existing commands or environments. Prefer standard LaTeX over new custom definitions.

In `sources3.tex`, provide the complete bibliographic information and URLs where appropriate. Organize the entries so that it is clear which claims or subsections each source supports. The sources file should be useful for checking the report, rather than being an unexplained list of links.

Do not place citations, bibliography entries, URLs or citation markers in `section3.tex`. Keep the source documentation in `sources3.tex`, as required by the report format.

If compilation is available, check that the files integrate with the supplied LaTeX setup and verify the resulting page count. If compilation is not available, state that the page count is an estimate rather than claiming it has been verified.

Return the three files as separate, clearly named downloadable files, not as one combined document.

Prompt 7 — Review phase of my Section 3 corrections

Task: Impartial Review of Annotations on a Technical Report

Act as an independent academic reviewer with expertise in programming-language type systems, Haskell, F#, JavaScript, TypeScript, and technical writing.

I have attached a PDF containing a section of my report, with annotations that I wrote to point out possible technical inaccuracies, unclear explanations, misleading wording, and presentation problems. Your task is **to evaluate my annotations impartially**, do not to assume that my criticisms are correct.

1. **Review every annotation individually**
   Inspect the PDF, including the text highlighted by each annotation and the surrounding context. Review every annotation you can actually access.
   For each annotation, provide:
   (a) **Location:** page number and the passage to which the annotation refers.

(b) **Verdict:** choose exactly one:
- **Agree:** the annotation identifies a genuine problem.
- **Partly agree:** the concern is valid, but the reasoning, wording, or proposed solution needs refinement.
- **Disagree:** the annotation is mistaken, unjustified, or contradicted by the report.

(c) **Reasoning:** explain why you reached that verdict. Distinguish technical correctness, conceptual accuracy, clarity, pedagogy, and stylistic preference.

(d) **Recommended correction:** suggest a concrete revision to the report or, if appropriate, a better version of my annotation.

(e) **Confidence:** indicate whether your conclusion is well established or depends on a technical detail that needs further verification.

Do not merely paraphrase my annotations. Independently evaluate whether their underlying claims are justified.

2. **Be especially careful with technical claims**
The report compares Haskell's native kind system with F# and TypeScript approaches to higher-kinded abstractions. In particular, it discusses:
- TypeScript's native generic type constructors and the absence of native higher-kinded polymorphism.
- Type-level constructs such as conditional types, `keyof`, indexed access types, and `infer`.
- A URI-based registry encoding of `Functor` and generic functions such as `twice`.
- The difference between native language features, programmer-defined encodings, and runtime behaviour.
- The erasure of TypeScript's type-level constructs during compilation.
- Comparisons with Haskell and F#.

Check whether the relevant statements are technically accurate and whether the examples support the claims made about them.
When a claim depends on compiler behaviour, verify it where possible. Do not claim that code compiles or produces a particular output unless you have verified it or clearly identify the claim as an inference.
Use authoritative, current sources for claims about language behaviour and library history. Cite external sources for factual claims that require verification. If you cannot verify a claim, state that limitation explicitly.

3. **Evaluate the quality of the proposed corrections**
Do not assume that removing material is always better than explaining it, or that adding an explanation is always better than removing material.
Consider the report's intended audience, the section's length, its academic purpose, and the fact that it compares three programming-language approaches.
For example, when an annotation says that a concept is introduced too abruptly, determine whether the best solution is to remove it, explain it briefly where it first appears, or add a separate explanation. Justify your recommendation.
Likewise, distinguish a genuinely misleading statement from wording that is merely less elegant than an alternative.
If an annotation identifies a real problem but proposes an unsuitable correction, agree with the criticism while recommending a better solution.

4. **Identify additional problems**
After reviewing my annotations, identify any important technical, conceptual, pedagogical, or presentation problems that I did not annotate.
Prioritise these issues by severity:
- **Major:** incorrect claims, misleading comparisons, invalid examples, or misunderstandings that undermine the central argument.
- **Moderate:** unclear reasoning, insufficient justification, or explanations that are likely to confuse the intended reader.

- **Minor:** stylistic, formatting, or wording improvements.

For each additional issue, identify the relevant passage, explain the problem, and propose a correction. Avoid inventing problems merely to make the review longer.

5. **Required output format**
   Structure your response as follows:
   A **Overall assessment** Briefly explain whether the annotations are generally sound, mixed, or mostly unjustified. Identify the most important findings.
   B **Annotation-by-annotation review** Use a table with these columns:
   — Annotation — Verdict — Reasoning — Recommended correction — ——————————
   —
   Give every annotation its own entry. Do not combine separate annotations into one verdict.
   C **Additional issues** List the important problems not covered by my annotations, ordered by severity.
   D **Prioritised recommendations** Give a short list of the changes that should be made first, followed by optional editorial improvements.
   E **Final judgement** State whether the annotations, taken together, would improve the report if followed. Explain which annotations should be accepted as written, which should be revised, and which should be rejected.

Important constraints

- Be independent and impartial. Do not favour my interpretation merely because I wrote the annotations.
- Do not defend the report merely because it is already written.
- Do not silently correct inaccuracies in your analysis: explain what is wrong and why.
- Do not invent annotations, quotations, compiler results, or source references.
- If a PDF annotation or its highlighted passage cannot be read, identify that limitation rather than guessing.
- Produce the entire report again, modifying only the parts that need to be changed.

Prompt 8 — Conclusion

Role You are my technical writing assistant for an Advanced Programming university project on Haskell kinds and their analogues in F#, JavaScript/TypeScript, and other programming languages.
Context You are writing the final chapter of a report whose previous sections have already investigated:

- Haskell's kind system and higher-kinded polymorphism.
- F#'s type system, its limitations, and workarounds for higher-kinded abstraction.
- JavaScript/TypeScript's type-level features and techniques for encoding higher-kinded abstractions.

Read the attached report and use its existing content as the sole basis for the conclusion.
Task
Write a concise, conclusive final chapter that brings the report to a natural close. Synthesize the main findings from the preceding sections, highlighting what the comparison reveals about the role of kinds and the different ways programming languages support, approximate, or restrict abstraction over type constructors. The conclusion should:

- Summarise the central findings without repeating the technical explanations already given.
- Draw together the most important similarities and differences between the languages discussed.
- Reflect on the trade-offs between native language support and alternative mechanisms or encodings, where these were established in the report.
- End with a clear final takeaway that directly addresses the project's overall goal.

Style and scope

Write for an advanced computer science student who has read the entire report. Maintain its terminology, tone, and level of technical detail. Be concise and analytical rather than merely listing the contents of each section. Do not introduce new concepts, arguments, examples, languages, or claims that require further explanation. Avoid generic statements, repetitive summaries, unnecessary subsections, and an extended introduction.

Sources

No external research or citations are required. Base every statement on the material already presented in the report. Do not add a bibliography or introduce new references. LaTeX and deliverables Return exactly one file: `conclusion.tex`, containing the complete LaTeX source for the final chapter. Reuse the report's existing macros, environments, and formatting conventions where appropriate. Do not redefine existing commands or introduce unnecessary macros. Do not modify any preceding section.

Length

Aim for approximately half a page in the report's existing layout. Two pages is an absolute hard maximum, not a target. Prefer a shorter conclusion if the main findings can be synthesised effectively in less space. Do not add content merely to increase the length. Before finalising, check that the conclusion follows from the report, adds a sense of closure, avoids unnecessary repetition, and respects the length limit.

Prompt 9 — Final review

Role

You are an extremely rigorous technical reviewer specialising in programming language theory, type systems, type constructors, kind systems, and the programming languages discussed in this report. Your task is to perform a comprehensive, critical review of my entire Advanced Programming project report.

Context

I will provide the complete report. Read it in its entirety before beginning the review, taking into account the relationships between sections, the terminology used throughout, and the report's overall objectives.

The report investigates Haskell's kind system and compares it with related concepts, limitations, and workarounds in F# and JavaScript/TypeScript.

Assume that the report has already undergone several rounds of revision. Do not assume that its current correctness is satisfactory: scrutinise every claim, explanation, example, comparison, and conclusion independently.

Task

Perform an exhaustive technical and editorial review of the entire report. **Identify every possible error, inaccuracy, misleading statement, ambiguity, unjustified claim, or other weakness you can find, even if doing so requires being extremely pedantic.**

Your objective is to maximise the likelihood of detecting problems before the report is submitted, not to minimise the number of suggested corrections.

Examine, at minimum, the following categories:

1. **Technical correctness:** Incorrect definitions, explanations, terminology, code examples, type signatures, inferred types, compiler behaviour, language features, and claims about what a language does or does not support.
2. **Conceptual precision:** Confusion between related but distinct concepts, imprecise explanations, missing qualifications, incorrect generalisations, and statements that are technically defensible under one interpretation but false under another.
3. **Comparisons between languages:** Unfair or inconsistent comparisons, false equivalences, overlooked differences, inaccurate claims about expressiveness, and failure to distinguish native language features from encodings, libraries, conventions, or workarounds.

4. **Logical consistency:** Contradictions within a section or between sections, conclusions that do not follow from the presented arguments, inconsistent terminology, and claims that conflict with examples elsewhere in the report.
5. **Misleading wording:** Sentences that could give the reader an incorrect impression even if they are not strictly false, overconfident claims, ambiguous phrasing, missing caveats, and simplifications that go too far for the intended academic level.
6. **Completeness and justification:** Important qualifications or distinctions missing from an explanation, claims that need further support, reasoning gaps, and statements that are presented as established facts without sufficient justification.
7. **Code and LaTeX:** Incorrect or misleading code snippets, syntax or typing issues, inconsistencies between code and its explanation, broken or inappropriate cross-references, citation problems, and other detectable LaTeX issues.

These categories are not exhaustive. Report any other issue that could reasonably affect the report's accuracy, clarity, rigour, or credibility.

Review methodology

- Examine the report sentence by sentence wherever possible. Do not limit yourself to an overview or a list of major issues.
- Consider whether each technical statement is correct under the precise conditions in which it is presented.
- Check whether examples genuinely demonstrate the claims made about them.
- Pay particular attention to absolute statements such as *always*, *never*, *only*, *cannot*, and *does not support*. Determine whether they require qualifications or exceptions.
- Distinguish genuine errors from oversimplifications, ambiguities, stylistic weaknesses, and optional improvements.
- Do not flag a statement merely because it could be worded differently. Explain the concrete problem or the plausible risk of misunderstanding.
- Do not invent errors to make the review appear more exhaustive. If a statement is correct but could reasonably be misinterpreted, explain that distinction.
- Do not silently resolve contradictions by choosing whichever interpretation seems most plausible. Identify the conflict explicitly.
- Do not assume that a claim is correct simply because it is cited, and do not assume that it is incorrect merely because it is not familiar to you.
- Where a claim depends on a particular language version, compiler, implementation, or set of assumptions, identify that dependency.
- Where relevant, verify technical claims against authoritative sources, preferably official language documentation, specifications, primary academic publications, or original research papers. Do not rely solely on search-result snippets or secondary summaries.
- Never fabricate sources, quotations, language features, or counterexamples. If you cannot verify an alleged problem, explicitly mark it as unverified rather than presenting it as a confirmed error.

Required output

Organise your review into two parts.

Part 1 — Detailed list of potential issues

List every issue you identify, including minor and potentially debatable ones. Do not impose an arbitrary limit on the number of findings, and do not omit a plausible issue simply because it is relatively insignificant.

For each finding, provide:

1. **ID:** A unique identifier, such as E01, E02, and so on.
2. **Location:** The section, subsection, page, and paragraph where possible.
3. **Original text:** Quote the exact sentence or the smallest relevant passage from the report.
4. **Category:** For example, technical error, misleading wording, ambiguity, missing qualification, unsupported claim, inconsistency, citation issue, or style.

5. **Assessment:** Explain precisely what the problem is, why it matters, and under what interpretation or conditions the statement becomes incorrect or misleading.
6. **Evidence or reasoning:** Provide a counterexample, a technical explanation, or an authoritative source supporting your assessment, as appropriate.
7. **Suggested correction:** Propose a minimal correction or clarification where possible. If several interpretations are possible, explain the alternatives rather than arbitrarily choosing one.
8. **Confidence:** Indicate whether the issue is confirmed, highly likely, plausible but debatable, or unverified.
9. **Importance:** Classify it as critical, major, moderate, or minor, based on its impact on technical correctness, reader understanding, or academic credibility.

Keep separate findings separate when they concern different problems, even if they occur in the same paragraph. Conversely, do not inflate the number of findings by repeatedly reporting the same underlying problem. When several passages share one issue, identify the common problem and list all affected locations.

When external sources are used, include enough bibliographic information and a direct link for me to verify them. Clearly distinguish what the source establishes from your own interpretation.

Part 2 — Overall assessment

After the detailed list, provide a concise synthesis of your findings, including:

- The most serious problems that should receive attention first.
- Any recurring patterns of imprecision or misunderstanding.
- Any sections or topics that deserve a second, deeper verification pass.
- Any important limitations of your review, including claims you could not independently verify.

Do not repeat the entire detailed list in this part.

Important constraints

Your job is to identify and explain potential problems, not to decide which ones I must correct. I will judge each finding myself and decide whether a correction is necessary. Therefore, err on the side of reporting a plausible issue with appropriate caveats rather than suppressing it because you consider it too minor or debatable.

Do not rewrite the report, produce a corrected version, or modify its files unless I explicitly ask you to do so in a later prompt.

Do not prioritise brevity over coverage. A long, detailed review is preferable to a short review that misses relevant problems. At the same time, ensure that every finding is specific, technically reasoned, and useful rather than vague or repetitive.

Finally, do not declare the report error-free merely because you have not found further issues. State what you actually checked and be transparent about the limits of your verification.

3 Claude context

You are an assistant for this project for the "Advanced Programming" course of the University of Pisa. This is an experimental course where Artificial Intelligence is not only encouraged but also mandatory for the grade. You will have to produce a project on the following topic:

3.1 C10 - Kinds in Haskell and Their Analogues in F# and JavaScript

3.1.1 Goal

Understand Haskell's notion of **kinds** (the "types of type constructors") and investigate to what extent similar ideas exist or can be emulated in **F#** and **JavaScript**, both at the language and library/design level.

3.1.2 Tasks

1. **Explain** Haskell's kind system (e.g. `*`, `* -> *`, higher-kinded types), showing examples with type constructors (`Maybe`, `Either`, `[]`) and how the compiler uses kinds to type-check type-level code.
2. **Discuss** how F#'s type system relates to Haskell's kinds (what is supported, what is missing, any workarounds for higher-kinded types), and propose patterns or abstractions in JavaScript (e.g. type-encoding with TypeScript or runtime conventions) that approximate kind-like reasoning.
3. **Document** the prompts you used with AI tools and describe how you checked the correctness of the explanations and code examples in all three ecosystems.

3.2 Previous knowledge

You can assume good knowledge of all paradigms of programming languages, as well as the languages themselves. However, if there are some technicalities typical of one language you will have to delve more into them.

3.3 Project structure

You will have to elaborate a project with the following structure:

- AI output.
- Document with relevant prompts.
- Discussion on how the generated output was validated.

The *AI output* is the report you will have to produce, later I will tell you how it is should be divided. This is the only one that you should focus on. The *Document with relevant prompts* is a document that I will produce by selecting the prompts I found to be the most relevant and interesting; given the fact that I will have to send even your outputs, they will also have to be well structured and without errors. Finally, the *Discussion on how the generated output was validated* is a report that I will have to create explaining how I avoided hallucinations and checked the AI output.

3.3.1 AI output

I want the AI output to be written in LaTex, it will be divided into these 4 sections that talk about the following:

1. Kinds: what are they and how they are used (examples), and their usefulness
2. Deep dive in F# type system, how are kinds treated there?
3. How to approximate kind-like reasoning in JavaScript/TypeScript?
4. Conclusion, final recap of everything.